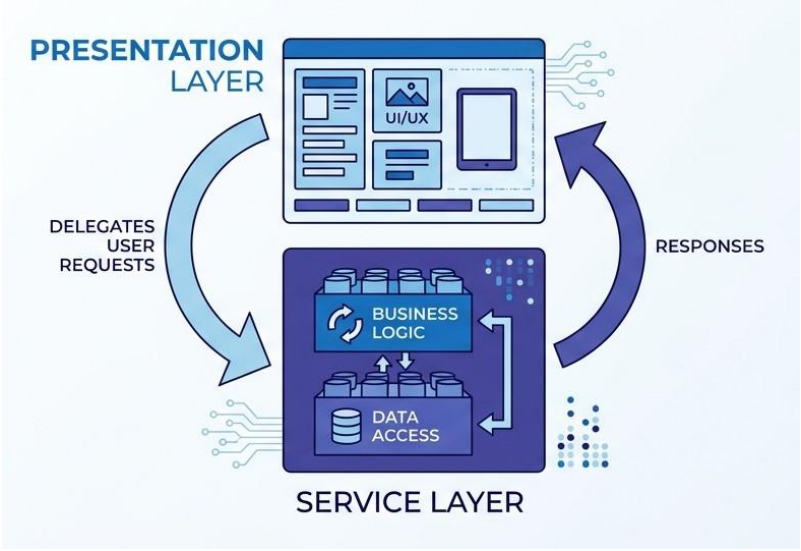
Introduction to Service Layer and Separating Controllers from Services in Spring Boot

Learning Outcomes

- **Differentiate Controller and Service Layer Responsibilities**
  Explain the core principle of separating HTTP handling from business logic.
- **Apply Software Design Principles (Cohesion & Coupling)**
  Evaluate software design choices by defining high cohesion and low coupling.
- **Usage of @Autowired annotation**
  Apply Spring's @Autowired annotation to automatically inject service layer dependencies into controller layer.
- **Develop and Test API endpoints**
  Build functional API endpoints backed by service layer data management and verify its behavior.



The Service Layer

Moving the business logic out of the controller

01

Controller's Job vs Service's Job

C

Controller Layer

THE WEB LAYER

- Receive HTTP requests. Return HTTP responses.
- Reads path variables and request bodies.
- Knows about URLs and status codes - and no business rules at all

That's it. Nothing else.

S

Service Layer

THE BUSINESS LAYER

- Executes business logic - assigning ids, finding, saving and deleting.
- Knows nothing about HTTP requests and HTTP responses
- Can be tested on its own, with no server running

The "how," not "what arrived over HTTP."

If business logic lives in the controller, the controller becomes untestable and unmaintainable.

Why This Two-Layer Split Matters

BEFORE

Everything in the Controller

- HTTP handling AND business logic tangled together
- Impossible to test the logic without a running server
- Every new rule means editing the same crowded class

AFTER

Controller Delegates to Service

- Controller just calls Service to execute business logic
- Business logic lives inside the Service class (Service Layer)
- Business logic is testable on its own, with no server needed
- Clean separation - each class (Controller class and Service class) has one clear job

Where Does This Code Belong ?

The brief

A colleague sends you five lines of code from a Spring Boot project and asks where each one should live: in the Controller, or in the Service?

1. Reject a product whose price is below zero
2. Spring boot read the id out of the URL with @PathVariable annotation
3. Give a new product the next available id
4. Spring boot return 404 Not Found error when nothing matches the id
5. Filter the catalogue down to products under £50

The verdicts

- Service - it is a business rule
- Controller - that is pure HTTP
- Service - the rule owns the ids
- Controller - 404 is a status code, its HTTP
- Service - filtering is logic

@Service – ProductService Class Creation

```
@Service
public class ProductService {

    private List<Product> products
        = new ArrayList<>();

    // business logic methods go here
}
```

- **@Service** marks the class as a Spring-managed business logic component
- The controller no longer touches the product list directly
- **private** product list is not visible to outside – Controller cannot see it
- Spring creates and manages the ProductService instance for you using component scanning

NOTE: ***@Autowired*** *wires this into the controller - we'll look at that shortly.*

Live Code – Reading Business Logic From the Service Class

```java
@Service
public class ProductService {

    private List<Product> products = new ArrayList<>();

    public List<Product> getAllProducts() {
        return products;
    }

    public Optional<Product> getProductById(Long id) {
        return products.stream()
                       .filter(p -> p.getId().equals(id))
                       .findFirst();
    }
}
```

What to notice

- Optional<Product> safely represents **“might not exist”** - no risky nulls
- .stream().filter().findFirst() scans the list for a match
- We won't unpack lambda syntax (->) in depth this week

Cohesion & Coupling

The vocabulary professional developers use to judge design

02

High Cohesion – One Job, Done Well - Single Responsibility Principle

✓

ProductService

ProductService manages **products specific business logics only** and **does nothing else**

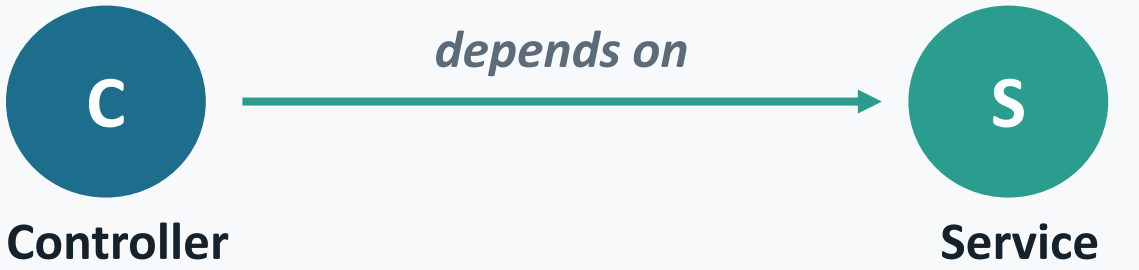
✗

A “kitchen-sink” service

Also sends emails, also parses files, also handles authentication - unrelated responsibilities crammed into one single class

High cohesion: a class does one thing well – Single Responsibility

Low Coupling – Minimal Dependencies



- *Few dependencies*
- *One direction only - the service has no idea the controller exists*

- **ProductController** depends on **ProductService** - it calls its public methods
- **ProductService does NOT** depend on **ProductController** - it never references it
- This is good coupling: dependencies point one way, and as little as possible

Good vs Bad Coupling

✓ Good Coupling

Controller → Service, one direction. Each class can change independently.

✗ Bad Coupling

Controller and Service both reach into the same shared logic directly - tangled, two-way dependencies.

Which one is easier to test in isolation, without starting the whole application?

Quick Check

Q1 **What does “high cohesion” mean?**

→ *A class does one job well - all of its responsibilities are closely related to that job.*

Q2 **What does “low coupling” mean?**

→ *Classes depend on each other as little as possible, and only in one direction.*

Q3 **Does ProductService need to know ProductController exists?**

→ *No. The service has no idea who calls it - that's exactly what makes the coupling good.*

@Autowired Surface Level

Today's shortcut for wiring dependencies

03

@Autowired – Automatically Wiring It Together

```java
@RestController
public class ProductController {

    @Autowired
    private ProductService productService;

    // endpoints go here
}
```

- Spring creates **ONE instance** of **ProductService**
- It hands that instance to the controller automatically
- @Autowired on the field is all that's needed for it to work

We'll understand exactly why this works in later. For now: Spring builds the object graph for you, and **@Autowired** just plugs it in to **Controller** class.

Building the Products API

04

From an empty service to four working CRUD endpoints.

Key Code Example - ProductService

```
@Service
public class ProductService {
    private List<Product> products
        = new ArrayList<>();
    private Long nextId = 1L;

    public List<Product> getAllProducts() {
        return products;
    }
    public Optional<Product> getProductById(
            Long id) {
        return products.stream()
            .filter(p ->
                p.getId().equals(id))
            .findFirst();
    }
    // continued →
```

```
    // ...ProductService (continued)

    public Product addProduct(
            Product product) {
        product.setId(nextId);
         nextId += 1L;
        products.add(product);
        return product;
    }

    public String deleteProduct(Long id) {
        boolean isDeleted =
         products.removeIf(p ->
            p.getId().equals(id));
        if (isDeleted) {
            return "Deleted successfully";
        } else {
            return "ID not found";
        }
    }
}
```

Wiring the Controller – Four Endpoints

```java
@RestController
@RequestMapping("/products")
public class ProductController {
    @Autowired
    private ProductService productService;

    @GetMapping()
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public Product getProductById(
            @PathVariable Long id) {
        return productService.getProductById(id)
       .orElse(null); // orElse(null) is a
// placeholder we replace with a real 404 later.
    }
    // continued →
```

```java
    // ...ProductController (continued)

    @PostMapping()
    public Product saveProduct(
            @RequestBody Product product) {
        return productService.addProduct(product);
    }

    @DeleteMapping("/{id}")
    public String deleteProductById(
            @PathVariable Long id) {
        productService.deleteProduct(id);
    }
}
```

GET /products GET /products/{id} POST /products DELETE /products/{id}

Controller to Service Delegation

ENDPOINT MAPPING

HTTP Method & Path	Controller Annotation	Delegated Service Call
GET /products	@GetMapping	productService.getAllProducts()
GET /products/{id}	@GetMapping + @PathVariable	productService.getProductById(id)
POST /products	@PostMapping + @RequestBody	productService.addProduct(product)
DELETE /products/{id}	@DeleteMapping + @PathVariable	productService.deleteProduct(id)

findByName() with a Stream Filter

```
public List<Product> findProductsByName(String name) {
    return products.stream()
                   .filter(p ->
                       p.getName().equalsIgnoreCase(name))
                   .toList();
}
```

A brief introduction to streams

Java Streams provide a simple and declarative way to process and manipulate collections of data efficiently.

Importance of Streams & Optional<> Data Handling in Spring Boot

DATA PROCESSING TIPS

Optional<>

- **Represents possible absence of a value** – Makes it explicit that a value may or may not exist.
- **Reduces null-related errors** – Helps avoid accidental exceptions. (NullPointerException)
- Safely represents a result that **might not exist**.
- **Improves API clarity** – Communicates to developers that a returned value may be unavailable.

Java Streams

- **Process collections efficiently** – Filter, sort, and transform data in a concise way.
- **Declarative programming** – Describe ***what*** should be done rather than ***how*** to do it step by step
- **Readable and maintainable code** – Reduces repetitive loops and boilerplate code.
- **Useful in Spring Boot applications** – Commonly used for processing data.
- **Supports functional-style operations** – Enables operations such as filter(), map() sorted() and etc.

Quick Check

Q1 **Why does getProductById return Optional<Product> instead of just Product?**

→ *It safely represents “might not be found” — without returning a risky null.*

Q2 **What does .stream().filter(...).findFirst() do here?**

→ *It scans the list, keeps only matching items, and returns the first match, if any.*

Q3 **Why can the controller and service be written independently and still work together?**

→ *Low coupling - the controller only needs to know the service's public methods, not its internals.*

The Journey of One Request – Save a Product

What actually happens when the browser sends POST /products

1. **The request arrives at the controller**
   Spring matches the URL and the HTTP method and turns the JSON body into a product object.

2. **The controller delegates immediately**
   One line: productService.addProduct(product). Controller does not decide anything itself.

3. **The service applies the business rules**
   It assigns the next id and stores the product in its private list.

4. **The service returns the saved object**
   Plain Java object all the way back. The service has no idea a browser is involved.

5. **The controller shapes the response**
   It hands the object to Spring, which serialises it to JSON and sets the status code and send it to browser.

RULE: Requests travel downwards and answers travel back up. The service never calls the controller - not once, not ever.

Test It Using Swagger UI and Postman

1. **Restart the application**

   Stop the running app and press Run again so the new endpoints load.

2. **POST /products with a JSON body**

   Send {"name": "Keyboard", "price": 59.99} and confirm a 200/201 response.

3. **GET /products**

   Confirm the returned array now includes the product you just created.

4. **GET /products/{id} and DELETE /products/{id}**

   Fetch it by id, then delete it, then GET again to verify it's gone.

Testing Workflow (Swagger UI / Postman)

API VERIFICATION

STEP 01

Restart App

Stop and re-run application to load new endpoints.

STEP 02

POST Product

Send JSON body

Eg:

```
{
"name": "Keyboard",
"price": 59.99
}
```

STEP 03

GET /products

Confirm returned array includes newly created product.

STEP 04

GET & DELETE

Fetch item by ID, delete it, and confirm removal.

References

- **Baeldung – Spring @Controller vs @RestController**
  - Explains how controllers receive HTTP requests and route them to business logic.

- **GeeksforGeeks – Spring Boot REST API Example**
  - Provides clear, step-by-step code snippets demonstrating how to build endpoints using @GetMapping, @PostMapping, @PathVariable, and @RequestBody.

- **GeeksforGeeks – Spring Boot Annotations (@Service, @Autowired)**
  - Explanations of how Spring manages dependencies automatically with @Autowired and bean stereotypes like @Service.

- **W3Schools – Java Streams Tutorial**
  - Includes interactive "Try It Yourself" code editors showing how filtering (.filter()), mapping, and finding elements (.findFirst()) work in functional Java.